Metadata of the chapter that will be visualized in SpringerLink

Book Title	Mathematical Optimization Theory and Operations Research	
Series Title		
Chapter Title	Using Merging Variables-Based Local Search to Solve Special Variants of MaxSAT Problem	
Copyright Year	2020	
Copyright HolderName	Springer Nature Switzerland AG	

Author	Family Name	**Otpuschennikov**
	Particle	
	Given Name	**Ilya V.**
	Prefix	
	Suffix	
	Role	
	Division	
	Organization	ISDCT SB RAS
	Address	Irkutsk, Russia
	Email	otilya@yandex.ru
	ORCID	http://orcid.org/0000-0001-5209-1803
Corresponding Author	Family Name	**Semenov**
	Particle	
	Given Name	**Alexander A.**
	Prefix	
	Suffix	
	Role	
	Division	
	Organization	ISDCT SB RAS
	Address	Irkutsk, Russia
	Email	biclop.rambler@yandex.ru
	ORCID	http://orcid.org/0000-0001-6172-4801

Abstract

In this paper we study the inversion of discrete functions associated with some hard combinatorial problems. Inversion of such a function is considered in the form of a special variant of the well-known MaxSAT problem. To solve the latter we apply the previously developed local search method based on the Merging Variables Principle (MVP). The main novelty is that we combine MVP with evolutionary strategies to leave local extrema generated by Merging Variables Hill Climbing algorithm. The results of computational experiments show the effectiveness of the proposed technique in application to inversion of several cryptographic hash functions and to one problem of combinatorial optimization, which is a variant of the Facility Location Problem.

Keywords

Pseudo-boolean optimization - SAT - MaxSAT - Local search - Evolutionary algorithms - Cryptographic hash functions - Branch Location Problem

Author Proof

Using Merging Variables-Based Local Search to Solve Special Variants of MaxSAT Problem

Ilya V. Otpuschennikov and Alexander A. Semenov(✉)

ISDCT SB RAS, Irkutsk, Russia
otilya@yandex.ru, biclop.rambler@yandex.ru

AQ1

Abstract. In this paper we study the inversion of discrete functions associated with some hard combinatorial problems. Inversion of such a function is considered in the form of a special variant of the well-known MaxSAT problem. To solve the latter we apply the previously developed local search method based on the Merging Variables Principle (MVP). The main novelty is that we combine MVP with evolutionary strategies to leave local extrema generated by Merging Variables Hill Climbing algorithm. The results of computational experiments show the effectiveness of the proposed technique in application to inversion of several cryptographic hash functions and to one problem of combinatorial optimization, which is a variant of the Facility Location Problem.

Keywords: Pseudo-boolean optimization · SAT · MaxSAT · Local search · Evolutionary algorithms · Cryptographic hash functions · Branch Location Problem

1 Introduction

In the present paper we study one particular class of the well-known MaxSAT problem [7]. The main feature of this class is that the combinatorial dimension of the considered problem can be significantly less than the number of variables that occur in a Boolean formula corresponding to this problem. Using this fact we describe a special strategy for solving MaxSAT from the mentioned class, which is built on the so-called Merging Variables Principle (MVP) proposed in [40]. The main goal of the present paper is to demonstrate the applicability of MVP to MaxSAT when the ratio between the combinatorial dimension of the considered problem and the number of variables in the corresponding Boolean formula is small.

In the next section we provide general information necessary for understanding the main results of the paper. Section 3 contains the main theoretical contribution of this paper. Namely, we introduce here a special class of MaxSAT problem, which is associated with the problems of finding preimages of quickly computable discrete functions. Essentially, we provide the theoretical basis that

© Springer Nature Switzerland AG 2020
Y. Kochetov et al. (Eds.): MOTOR 2020, CCIS 1275, pp. 1–16, 2020.
https://doi.org/10.1007/978-3-030-58657-7_29

Author Proof

allows us to apply MVP to the specified class of MaxSAT. Section 4 contains a short description of Merging Variables Principle as well as a description of several strategies used to exit from strong local extrema points. In Sect. 5 we present the main experimental contribution of the paper. We consider two families of MaxSAT benchmarks which can be attributed to the class introduced in Sect. 3. We demonstrate that our algorithms (based on MVP) outperform some of the best solvers that are usually applied to such problems. In Sect. 6 we briefly summarize the main results and talk about our future work.

2 Preliminaries

The variables that take values from the set $\{0, 1\}$ are called Boolean. A Boolean formula (or a propositional formula) is an expression constructed according to special pre-defined rules over the alphabet that includes Boolean variables and special symbols called Boolean connectives (according to [29]). For an arbitrary $k \in \mathbb{N}$ let us denote by $\{0, 1\}^k$ the set of all binary words (Boolean vectors) of length k. We refer to $\{0, 1\}^k$ as to a Boolean hypercube or a Boolean cube.

Let $X = \{x_1, \ldots, x_k\}$ be a set of Boolean variables. Then we can consider an arbitrary Boolean vector $\alpha \in \{0, 1\}^k$ as an assignment of variables from X (assuming that there is a fixed mapping between the coordinates of α and the variables from X). Now let F be a Boolean formula over the set of Boolean variables $X = \{x_1, \ldots, x_k\}$ and $\alpha \in \{0, 1\}^k$ be an arbitrary assignment of variables from X. We can define in a standard manner (according to [10]) a substitution of α into F. Boolean Satisfiability Problem (SAT) is formulated as the following question: if there exists such an $\alpha \in \{0, 1\}^k$, the substitution of which to F results in true. Below we will refer to it as $F(\alpha) = 1$. If such an α exists, then F is called satisfiable, otherwise – unsatisfiable. It is commonly known that SAT for any Boolean formula F can be reduced in polynomial time to SAT for formula in Conjunctive Normal Form (CNF) using Tseitin transformations [41].

SAT is a classical NP-complete problem [11]. It is often the case in practice that one not only needs to determine the satisfiability of CNF C, but also to find its satisfying assignment if C is satisfiable. In this formulation SAT is an NP-hard problem [17]. Despite being so hard in the worst-case scenario, SAT can be solved quite effectively for a wide class of application.

MaxSAT is the optimization variant of SAT. Traditionally, this problem is formulated as follows. Given a CNF C over a set of Boolean variables $X = \{x_1, \ldots, x_k\}$ to find as assignment $\alpha \in \{0, 1\}^k$ of variables from X such that the number of clauses satisfied by α in C is maximal. It is clear that α is a satisfying assignment of C if and only if it satisfies all clauses in C. Thus, MaxSAT is NP-hard. During the last 10 years there have been published a huge number of papers that study different aspects of MaxSAT. One of the main strategies for solving MaxSAT consists in iteratively invoking a SAT solver as an oracle [2,12,16,20–22,32,35], etc. Usually, a SAT solver based on a CDCL algorithm [28] is used for this purpose.

In the present paper we describe the algorithms for solving MaxSAT that are based on a completely different foundation. In particular, we consider MaxSAT

Author Proof

as a maximization problem for a pseudo-Boolean functions of a special kind and solve it using a variant of local search methodology. Note that the basic idea of the corresponding approach is far from being novel: in this context we would like to cite an extensive review [19] and the related citations. The novelty of the approach proposed in the present paper is formed by the following components:

1. We work with a class of CNFs for which it is possible to significantly reduce the search space over which the local search operates when solving MaxSAT.
2. We combine the well-known Hill Climbing algorithm, as a basic local search scheme, with Merging Variables technique described in [40].

Hereinafter, consider the maximization problem for a pseudo-Boolean function, i.e. a function of the following kind [8]

$$f : \{0,1\}^n \to \mathbb{R}. \tag{1}$$

There is a large number of problems, in which little to nothing is known about the analytical properties of a function f. Below we assume that the only thing we know for sure about f is that for an arbitrary $\alpha \in \{0,1\}^n$ we can effectively compute the value $f(\alpha)$. In other words, we suppose that for each $\alpha \in \{0,1\}^n$ the value $f(\alpha)$ is produced by an oracle O_f. In these conditions, the search for a maximum of function (1) transforms into a problem of traversal of $\{0,1\}^n$ in accordance with some rational scenario. Usually, one of the most used scenario of such kind is the local search. To use local search it is necessary to specify a neighborhood function [9]

$$\aleph : \{0,1\}^n \to 2^{\{0,1\}^n}, \tag{2}$$

which defines some neighborhood structure on $\{0,1\}^n$. For an arbitrary $\alpha \in \{0,1\}^n$ the value of (2) for this α is denoted by $\aleph(\alpha)$ and is called the neighborhood of α.

The simplest example of the strategy for maximization of (1) that is based on the local search scenario can be given by the algorithm, which is often referred to as Hill Climbing (HC) [37]. Below we present the pseudocode of HC.

Input: an arbitrary point $\alpha \in \{0,1\}^n$, a value $f(\alpha)$;

1. α is a current point;
2. traverse the points from $\aleph(\alpha)\backslash\{\alpha\}$, computing for each point α' from this set a value $f(\alpha')$ (using the oracle O_f). If there is such a point α' that $f(\alpha') > f(\alpha)$, then go to step 3, otherwise go to step 4;
3. $\alpha \leftarrow \alpha'$, $f(\alpha) \leftarrow f(\alpha')$, go to step 1;
4. $\alpha^* \leftarrow \alpha'$; $(\alpha^*, f(\alpha^*))$ is a local maximum of (1) on $\{0,1\}^n$;

Output: $(\alpha^*, f(\alpha^*))$.

The neighborhoods in $\{0,1\}^n$ (or in any other search space) can be defined using different methods. One of the simplest is to use the Hamming neighborhoods in $\{0,1\}^n$: for an arbitrary $\alpha \in \{0,1\}^n$ its Hamming neighborhood of

Author Proof

radius R is the following set $\aleph_R(\alpha) = \{\alpha' | d_H(\alpha, \alpha') \leq R\}$, where by $d_H(\alpha, \alpha')$ we mean the Hamming distance between vectors α and α' [27].

Now let us consider an arbitrary CNF C over a set of Boolean variables $X = \{x_1, \ldots, x_k\}$. Recall, that C is a formula of the following kind: $C = D_1 \wedge \ldots \wedge D_m$, where D_j, $j \in \{1, \ldots, m\}$ are the Boolean formulas called clauses. For example, the CNF $(x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$ is defined over the set $X = \{x_1, x_2, x_3\}$ and contains 3 clauses. With an arbitrary CNF C over $X = \{x_1, \ldots, x_k\}$, which consists of m clauses let us associate the following function of the kind (1):

$$f_C : \{0,1\}^k \rightarrow \{0, 1, \ldots, m\} \tag{3}$$

For an arbitrary $\alpha \in \{0,1\}^k$ the value $f_C(\alpha)$ is the number of clauses satisfied by the assignment α. Thus, an arbitrary C is satisfiable if and only if $max_{\{0,1\}^k} f_C = m$. The MaxSAT problem consists in the following: for an arbitrary CNF C over the set of Boolean variables X, $|X| = k$ to maximize the function of the kind (3) over a Boolean hypercube $\{0,1\}^k$.

3 On One Special Case of MaxSAT Problem

In this section we introduce such a subclass of CNFs for which it is possible to effectively move from the maximization problem for (3) over $\{0,1\}^k$ to the maximization problem for a special function

$$g_C : \{0,1\}^q \rightarrow \{0, 1, \ldots, m\} \tag{4}$$

over $\{0,1\}^q$, where $q << k$. To better describe the proposed approach, we will require some additional comments.

As we noted above, both SAT and MaxSAT are NP-hard problems. It means that a wide spectrum of combinatorial problems can be reduced to them. When constructing such a reduction it is often necessary to introduce a large number of new variables that play an auxiliary role and are not directly connected to an original combinatorial problem. Below we describe in more detail one class of CNFs related to mentioned situation.

Consider an arbitrary algorithm that transforms binary words of length q into binary words of length u. Such an algorithm specifies (or computes) a discrete function of the kind

$$h : \{0,1\}^q \rightarrow \{0,1\}^u. \tag{5}$$

Given an arbitrary $\gamma \in Range\ h \subseteq \{0,1\}^u$ the problem of inversion of h is to find such $\beta \in \{0,1\}^q$, that $h(\beta) = \gamma$. Many cryptanalysis problems can be viewed as inversion problems of functions that are specified by corresponding cryptographic algorithms.

The corollary of the well known Cook-Levin theorem [11,18] is the fact that for an algorithm A_h defining h of the kind (5) it is possible to effectively construct CNF C_h, that contains all the information about how A_h works on arbitrary inputs from $\{0,1\}^q$. The technique used to construct CNFs of the kind C_h

Author Proof

contains two stages: on the first stage a Boolean circuit S_h over an arbitrary complete basis is constructed (usually a $\{\wedge, \neg\}$-basis is used). Then for circuit S_h a CNF C_h is constructed using the so-called Tseitin transformations [41]. In the process of these transformations, the inputs, outputs and inner nodes of circuit S_h are associated with Boolean variables. In particular, with the input of S_h the Boolean variables forming the set $X^{in} = \{x_1, \ldots, x_q\}$ are associated. The remaining variables are auxiliary. The set of auxiliary variables contains the set $Y = \{y_1, \ldots, y_u\}$ formed by the variables corresponding to the outputs of circuit S_h. For an arbitrary $\gamma \in \{0,1\}^u$ let us denote by $C_h(\gamma)$ the CNF, formed from C_h as a result of substituting γ into C_h (as an assignment of variables from Y).

Below we will use the notion of Strong Backdoor Set first introduced in [43] in the context of Constraint Satisfaction Problem (CSP). Let us provide the definition of this notion in relation to SAT which is a special case of CSP.

Let C be an arbitrary CNF over the set of Boolean variables X and let B be an arbitrary subset of X. Suppose that β is an arbitrary assignment of variables from B and let us denote this fact as $\beta \in \{0,1\}^{|B|}$. Denote by $C[\beta/B]$ the CNF obtained from C as a result of substituting a Boolean vector β into C (as an assignment of variables from B) and performing all possible elementary transformations.

Definition 1 (see [43]). *Let C be an arbitrary CNF formula over a set of variables X, and let A be a polynomial-time algorithm. A non-empty set B, $B \subseteq X$, is a Strong Backdoor Set for C w.r.t. algorithm A if for each $\beta \in \{0,1\}^{|B|}$ algorithm A results an answer for SAT of the CNF $C[\beta/B]$.*

Note that the trivial SAT solving algorithm for an arbitrary CNF C over X consists in traversing all possible vectors from $\{0,1\}^{|X|}$, substituting them into C and computing the value of a Boolean function defined by C. Let us call such an algorithm a brute force algorithm. It is easy to see that its complexity is $2^{|X|} \cdot p(|C|)$, where $p(\cdot)$ is a polynomial and $|C|$ is the length of CNF C description. Due to the Definition 1, if B is a Strong Backdoor Set B: $|B| << |X|$, then there exists an algorithm for solving SAT for C with the complexity of $2^{|B|} \cdot q(|C|)$, where $q(\cdot)$ is some polynomial. In this algorithm for each assignment $\beta \in \{0,1\}^{|B|}$ of variables from B the polynomial algorithm A is invoked in application to CNF $C[\beta/B]$. The complexity of the resulting algorithm may be significantly lower than the complexity of the brute force algorithm.

The important fact related to arbitrary CNFs of the kind $C_h(\gamma)$ consists in that the set X^{in} is the Strong Backdoor Set w.r.t a simple algorithm called the Unit Propagation rule (UP) [14,28].

Recall that an arbitrary literal over X is a formula of the kind $l_\lambda(x)$, $x \in X$, $\lambda \in \{0,1\}$, where:

$$l_\lambda(x) = \begin{cases} \neg x, \ \lambda = 0 \\ x, \ \ \lambda = 1 \end{cases}$$

The Unit Propagation rule in application to an arbitrary CNF C over X and a set of literals $L = \{l_{\lambda_1}(x'_1), \ldots, l_{\lambda_s}(x'_s)\}$, $\{x'_1, \ldots, x'_s\} \subseteq X$ works as follows.

Author Proof

Consider a CNF

$$l_{\lambda_1}(x'_1) \wedge \ldots \wedge l_{\lambda_s}(x'_s) \wedge C,$$

for an arbitrary $j \in \{1, \ldots, s\}$ first all clauses in C that contain $l_{\lambda_j}(x'_j)$ are removed, and from each clause containing the literal $\neg l_{\lambda_j}(x'_j)$ this literal is removed. Note that as a result of UP application it is possible to derive new unit clauses, i.e. the clauses that contain a single literal. Then one can apply the Unit Propagation rule to these newly derived literals, etc. In [14] there was proposed the algorithm implementing UP that has a linear complexity in the size of description of C. Below we employ the following important fact.

Lemma 1. *Let C_h be a CNF over a set of Boolean variables X, constructed according to the method outlined above based on the algorithm specifying function h of the kind (5). Let $\beta = (\beta_1, \ldots, \beta_q)$ be an arbitrary assignment of variables from $X^{in} = \{x_1, \ldots, x_q\}$. Then the result of application of only the Unit Propagation rule to CNF*

$$l_{\beta_1}(x_1) \wedge \ldots \wedge l_{\beta_q}(x_q) \wedge C_h \tag{6}$$

is the derivation of the values of all variables from $X \backslash X^{in}$ including an assignment of variables from $Y = \{y_1, \ldots, y_u\}$ such that $y_1 = \gamma_1, \ldots, y_u = \gamma_u$ and $h(\beta) = \gamma$, $\gamma = (\gamma_1, \ldots, \gamma_u)$.

Definition 2. *Assume that all conditions of Lemma 1 are satisfied and α is an assignment of variables from X, which was derived as the result of application of UP to CNF (6). Then we will say that α is induced by β.*

The statements that are close to the Lemma 1 in spirit have been independently proven in several papers [5,38]. The direct corrolary of Lemma 1 is the fact that for an arbitrary CNF of the kind $C_h(\gamma)$ constructed for an algorithm defining a function $h : \{0,1\}^q \to \{0,1\}^u$, the set X^{in} is a Strong Backdoor Set w.r.t. Unit Propagation rule. Any set of this kind is referred to as Strong Unit Propagation Backdoor Set (SUPBS).

Once again, let C_h be a CNF over a set of Boolean variables X constructed for the algorithm defining a function of the kind (5). Fix an arbitrary $\gamma \in \{0,1\}^u$ and consider CNF $C = C_h(\gamma)$. Assume that the number of clauses in C is m. Let us introduce the following function

$$g_C : \{0,1\}^q \to \{0, 1, \ldots, m\} \tag{7}$$

defined as follows. For an arbitrary $\beta \in \{0,1\}^q$, $\beta = (\beta_1, \ldots, \beta_q)$, consider it as an assignment for variables from $X^{in} = \{x_1, \ldots, x_q\}$ and construct a CNF (6). Then apply to this CNF the Unit Propagation rule. Let α be an assignment of all variables from X, that is induced by β. Then we define $g_C(\beta)$ as the number of clauses in C that are satisfied by vector α. The following fact holds.

Theorem 1. *Consider an arbitrary function h of the kind (5) and CNF C_h constructed using the algorithm specifying h. Let us fix an arbitrary $\gamma \in$ Range $h \subseteq$*

Author Proof

$\{0,1\}^u$ and consider CNF $C = C_h(\gamma)$. Assume that $X : |X| = k$ is the set of variables occurring in C. Consider two problems: in the first one we need to maximize the function f_C of the kind (3) over $\{0,1\}^k$. In the second the goal is to maximize function g_C of the kind (7) over $\{0,1\}^q$. Let $\beta^* = argmax_{\{0,1\}^q} g_C$ and α^* be an assignment of variables from X induced by β^*. Then $\alpha^* = argmax_{\{0,1\}^k} f_C$.

Proof (Sketch proof). The key aspect of the proof consists in the fact that γ is chosen from the range of the function h. In this case it follows from the Cook-Levin theorem and the properties of Tseitin transformations that CNF $C = C_h(\gamma)$ is satisfiable. The detailed description of this fact can be found in [39]. Thus, in the considered case there exists an assignment that satisfies C and, therefore, $max_{\{0,1\}^k} f_C = m$. Again, using the technique described in [39] it is possible to show that each assignment α that satisfies C corresponds to exactly one Boolean vector $\beta \in \{0,1\}^q$ that induces α in the sense of Definition 2. This fact means that the Theorem 1 holds.

As a conclusion of this section, we would like to note that in the context of the proposed approach we can view the problem of inversion of an arbitrary function (5) as a special variant of the MaxSAT problem. Essentially, we maximize the number of satisfied clauses in a CNF $C_h(\gamma)$, but thanks to the above it is possible to work only with the vectors from the Boolean hypercube $\{0,1\}^q$, considering them as assignments of variables from SUPBS X^{in}. Note that in the cases that are interesting from the practical point of view, X^{in} may contain tens or hundreds of variables while set X over which CNF $C_h(\gamma)$ is defined may consist of tens of thousand variables.

4 Merging Variables Principle

Merging Variables Principle (MVP) was proposed in the paper [40], though closely related approaches appeared in much earlier works [1,3]. MVP is aimed at pseudo-Boolean optimization. It is based on a very simple idea: as it was shown in [40], one can effectively construct a one-to-one correspondence between $\{0,1\}^n$ and a special metric space, which is denoted as D^μ. Let us connect with $\{0,1\}^n$ the set of Boolean variables $X = \{x_1, \ldots, x_n\}$, and let the set of variables $Y = \{y_1, \ldots, y_r\}$, $1 \leq r < n$ be connected with D^μ. Additionally, we assume that some surjection mapping $\mu : X \to Y$ (merging mapping) is specified. Then

$$D^\mu = D_1 \times \ldots \times D_r$$

where D_j, $j \in \{1, \ldots, r\}$ are domains for variables from Y. Let us establish a one-to-one correspondence

$$\tau_\mu : D^\mu \to \{0,1\}^n$$

between D^μ and $\{0,1\}^n$. Finally, let us move from maximization of pseudo-Boolean function $f : \{0,1\}^n \to \mathbb{R}$ to maximization of μ-conjugated function $F_{f,\mu} : D^\mu \to \mathbb{R}$ (see [40]). We can define the Hamming metric over space D^μ and

Author Proof

specify a neighborhood structure formed by neighborhoods of radius 1. Actually for an arbitrary μ the corresponding neighborhood structure in D^{μ} generates special neighborhood structure in $\{0,1\}^n$ and the latter is formed by the neighborhoods which are, generally speaking, not Hamming neighborhoods. It is easy to construct examples when f reaches a local maximum in $\alpha \in \{0,1\}^n$, but $\tau_{\mu}^{-1}(\alpha)$ is not local maximum of $F_{f,\mu}$ in D^{μ}. However, the following holds: $max_{\{0,1\}^n} f = max_{D^{\mu}} F_{f,\mu}$. In [40] we described the Hill Climbing algorithm augmented with Merging Variables strategy, to which we further refer as Merging Variables Hill Climbing (MVHC). Also, in [40] it was noted that we can view MVP as a special variant of the well known Variable Neighborhood Search strategy [31]. Using MVP we can effectively generate many different neighborhood structures in $\{0,1\}^n$.

An arbitrary merging mapping μ can be constructed as an example of classic distribution of particles to boxes. In all computational experiments we use the so-called uniform merging mapping ([40], definition 7). In this case for a fixed r we pursue the goal to obtain r boxes which are filled by particles almost uniformly. We achieve this by filling the boxes one-by-one: the first box on the first step, the second box on the second step, and so on. As a result we have the situation when the number of variables in each box is either $\lfloor n/r \rfloor$ or $\lfloor n/r \rfloor + 1$. Each box represents the set of preimages (w.r.t μ) of a corresponding variable from Y. Let us denote the maximum number of preimages for variables from Y as l. As it was noted in [40], MVHC with uniform merging mapping can be naturally implemented in form of multi-thread application: distinct threads process distinct neighborhood fragments which correspond to domains of specific merged variables. In this case all the threads will perform almost equal amount of work because all the domains have almost equal power.

Definition 3. *Assume that MVHC found such an $\alpha^* \in \{0,1\}^n$ that each $\beta_i = \tau_{\mu_i}^{-1}(\alpha^*), i \in \{1,\ldots,t\}$ is a local maximum of F_{f,μ_i} in D^{μ_i}. Let us refer to such an α^* as strong local maximum of f w.r.t. mappings μ_i, $i \in \{1,\ldots,t\}$.*

In the experiments presented in the next section we use several evolutionary strategies to escape strong local maxima to which MVHC falls in. Let us give a short description of these algorithms. The most simple example of such strategy is the (1 + 1)-Evolutionary Algorithm ((1 + 1)-EA) [33]. This is a simple algorithm based on the concept of a random mutation. In the case of (1 + 1)-EA a single random mutation of an arbitrary $\alpha \in \{0,1\}^n$ consists in a series of n independent Bernoulli trials with probability $p = \frac{1}{n}$. If i, $i \in \{1,\ldots,n\}$ is the number of successful trial, then the bit number i in α is flipped (changed from 0 to 1 or from 1 to 0). (1 + 1)-EA is extremely ineffective in theory [15], but often shows quite good performance in practice. As it was noted in [42], this can be attributed to the fact that on average (1 + 1)-EA acts in a fashion similar to Hill Climbing (because the expected value of the number of flipped bit after a single random mutation is 1). At the same time, unlike Hill Climbing, (1 + 1)-EA can move from α to any other point from $\{0,1\}^n$ with a non-zero probability.

There are several techniques that make it possible to reduce the worst-case estimation of (1 + 1)-EA effectiveness (in the context of the complexity measure

Author Proof

introduced in [15]) by changing the mutation rate in its base schema. One of such algorithms is the $(1+1)-FEA_\beta$ described in [13]. This algorithm is based on the idea of employing the heavy-tailed mutation operator: such an operator flips an arbitrary bit in α with probability $\frac{\lambda}{n}$ instead of $\frac{1}{n}$ in case of the standard $(1+1)$-EA. Here, λ is a value of random variable that has the so-called Power-law distribution $D^{\beta}_{\frac{n}{2}}$ with the parameter β [13]. The worst case estimation of this algorithm is $O(n^\beta \cdot 2^n)$ for any pseudo-Boolean function instead of n^n in case of the standard $(1+1)$-EA. In our computational experiments we used the $(1+1)-FEA_\beta$ variant with $\beta = 3$ to exit local extrema since in this case (according to [13]) the expected value of the number of flipped bits in one random mutation is $b \to 1.3685\ldots$ when $n \to \infty$ (for $\beta \leq 2$ this value grows to infinity with the increase of n).

We also performed the generation of a new starting point based on several strong local maxima, using a special variant of a genetic algorithm described in [36]. In this algorithm a population is the set of different strong local maxima constructed by MVHC during its work. Denote a current population by $P_{current}$ and a new population by P_{next}. Hereinafter, $|P_{current}| = |P_{next}| = N$ for some fixed N. Assume that $P_{current} = \{I_1, \ldots, I_N\}$. Let us associate with $P_{current}$ the distribution $D_{current} = \{p_1, \ldots, p_N\}$, where $p_i = \frac{f(I_i)}{\sum_{i=1}^{N} f(I_i)}$, $i \in \{1, \ldots, N\}$, where f is our objective function. A new population P_{next} is formed as a result of several actions. First, choose from $P_{current}$ E individuals with maximal weight (the value of f) and add them to P_{next} (it corresponds to the so-called elitism concept [26]). At the second step, choose from $P_{current}$ H individuals in accordance with the distribution $D_{current}$ and apply to each of them the standard $(1+1)$-random mutation by flipping each bit with probability $\frac{1}{n}$. The resulting individuals are added to P_{next}. Finally, choose individuals from $P_{current}$ according to the fixed distribution and perform standard two-point crossover to them [26]. The G individuals formed that way are also added to P_{next}. It is necessary that $E + H + G = N$.

The decision to generate a new starting point is made when we have s strong local maxima w.r.t t uniform merging mappings $\mu_1, \ldots, \mu_t$. The parameters s and t are tuned in the course of testing. Assume that using one of the methods outlined above we generated a new point α'. Then, in accordance to the ideas from [40] the point α' is checked to see if it is located at the Hamming distance $\geq l+1$ from the current set of strong local maximums. If this condition is held, then α' is taken as a new starting point, otherwise the process of generation α' is repeated.

5 Computational Experiments

In our computational experiments we considered two classes of tests. The first class is related to problems of finding preimages of several known cryptographic hash functions. In particular, we studied the MD-5, SHA-1 and SHA-256 hash functions. According to [30] a cryptographic hash function is a function of the kind $\chi : \{0,1\}^* \to \{0,1\}^c$, where $\{0,1\}^*$ is a Kleene star over the set $\{0,1\}$

Author Proof

and c is a constant. Preimage attacks for χ implies finding a preimage of some hash value from $\{0,1\}^c$. Usually, one searches for a preimage of a fixed length which as a rule of thumb coincides with the size of a single block of an input message. For the MD5, SHA-1 and SHA-256 hash functions this size is equal to 512 bits, and c is 128 for MD5, 160 for SHA-1 and 256 for SHA-256. For all considered functions there are no known algorithms that could find a preimage (e.g. a 512-bit input block) for a given hash image in realistic time. We can consider the problem of finding a 512-bit input the hash value of that contains a particular number of first zero bits. In this case the hardness of the resulting problems depends on the number of the zero bits. This very argumentation is used to justify the resistance of the majority of today's cryptocurrency protocols [34]. The problems of finding 512-bit inputs, the hash images of which contain K first zero bits, formed the first class of considered tests. Therefore, the name of the test is name of `function_K_launch_number` (for example: `MD5_20_3`).

In Table 1 column named msl-MV corresponds to regular MVHC with parameter l (the maximum number of preimages for variables from Y); msl-FEA corresponds to MVHC, complemented by strategies for exiting from strong local maxima, based on (1 + 1)-Fast Evolutionary Algorithm; msl-GA corresponds to the MVHC variant, which uses the Genetic Algorithm described above to exit strong local maxima.

We compared the efficiency of algorithms based on MVP with two multi-threaded SAT solvers Plingeling [6] and Painless [24] (`Pling` and `Painl` in Table 1 respectively). For a more detailed presentation of the results, we built cactus plots in the style which is commonly used in the SAT Race and SAT Competition competitions. To build cactus plots we used the mkplot tool[1].

It should be noted that some tests have not been solved in a time limit, which was equal to 5000 s as standard value (to each such a fact in Table 1 corresponds an empty cell). In the construction of cactuses, we take this into account by applying the PAR scoring system 2 used in the SAT Race and SAT Competition[2]: i.e. for an instance, which has not been resolved in the time limit, a decision time was chosen as a double time limit. Also the results of this series of experiments we demonstrate in Fig. 1. To plot the lines we take all runtimes of the corresponding algorithm on the considered instances that are below the time limit, order them in accordance with the increasing value (individually for each line) and plot the corresponding graph. It is commonly referred to as cactus plot.

As a final remark, we would like to note that the best results on the considered class of tests in accordance with the described form of presentation showed MVHC, complemented by strategies for exiting from strong local maxima, based on (1 + 1)-Fast Evolutionary Algorithm $(1+1)-FEA_\beta$ with $\beta = 3$. Each of the tested algorithms was launched in parallel mode on one node of the computing cluster of "Academician V.M. Matrosov"[3], where each node is equipped with

[1] https://github.com/alexeyignatiev/mkplot.
[2] http://sat-race-2019.ciirc.cvut.cz/.
[3] http://hpc.icc.ru.

Author Proof

Table 1. Results of solving inversion problems of cryptographic hash functions

Test	ms8-MV	ms8-GA	ms8-FEA	ms10-MV	ms10-GA	ms10-FEA	Pling	Painl
MD5_18_1	349	219	1380	803	524	164	-	905
MD5_18_2	215	57	384	269	49	87	2108	294
MD5_18_3	122	86	52	238	92	69	765	3036
MD5_20_1	40	1407	2093	1226	1521	175	3479	–
MD5_20_2	757	46	585	233	777	512	–	–
MD5_20_3	603	121	1782	450	670	1772	–	–
MD5_22_1	555	1143	611	940	–	–	–	–
MD5_22_2	1278	4998	–	651	4743	2662	–	–
MD5_22_3	2563	478	–	2079	3888	389	3425	–
SHA-1_18_1	117	200	267	95	497	585	190	1407
SHA-1_18_2	123	765	730	703	136	199	1061	3080
SHA-1_18_3	327	128	78	450	503	279	855	3077
SHA-1_20_1	273	2949	282	1930	538	779	537	288
SHA-1_20_2	69	1032	2488	511	2139	267	1007	1124
SHA-1_20_3	76	2004	4390	826	70	1595	2602	483
SHA-1_22_1	1204	–	1304	–	1654	925	2950	–
SHA-1_22_2	–	–	–	–	1150	4119	–	–
SHA-1_22_3	–	2973	2720	–	–	3629	–	1954
SHA-2_18_1	89	2396	330	26	26	26	44	3253
SHA-2_18_2	94	696	112	26	26	26	204	1994
SHA-2_18_3	220	201	233	26	26	26	763	2750
SHA-2_20_1	729	518	890	–	217	481	3637	–
SHA-2_20_2	125	1903	3101	105	720	4386	3211	–
SHA-2_20_3	751	124	1239	838	2913	1267	2147	414
SHA-2_22_1	–	497	3881	335	–	2039	–	–
SHA-2_22_2	–	–	–	4168	642	2400	–	–
SHA-2_22_3	4042	–	–	4487	–	2662	2290	–

two 18-core Intel Xeon E5-2695 CPUs. The results of experiments show that the best multithreaded solvers based on the CDCL algorithm (Plingeling and Painless), on the considered class of tests, significantly lost against all variants of Merging Variables Hill Climbing.

The second class of tests for which computational experiments were carried out was formed by MaxSAT variants of the Branch Location Problem (BLP), which can be considered as a special case of the Facility Location Problem [23]. The original formulation of the BLP uses weighted undirected complete bipartite graph $G(U, V, E, w)$, where U is a set of customers, V is a set of facilities, $E = U \times V$ is a set of edges. Function $w : E \to \mathbb{R}_{\geq 0}$ defines edge weights for this graph. The number of facilities, that should be deleted is denoted by $k, k \leq |V|$. Also the parameter Δ, $\Delta \geq 0$ is introduced. This parameter is a threshold by which

Author Proof

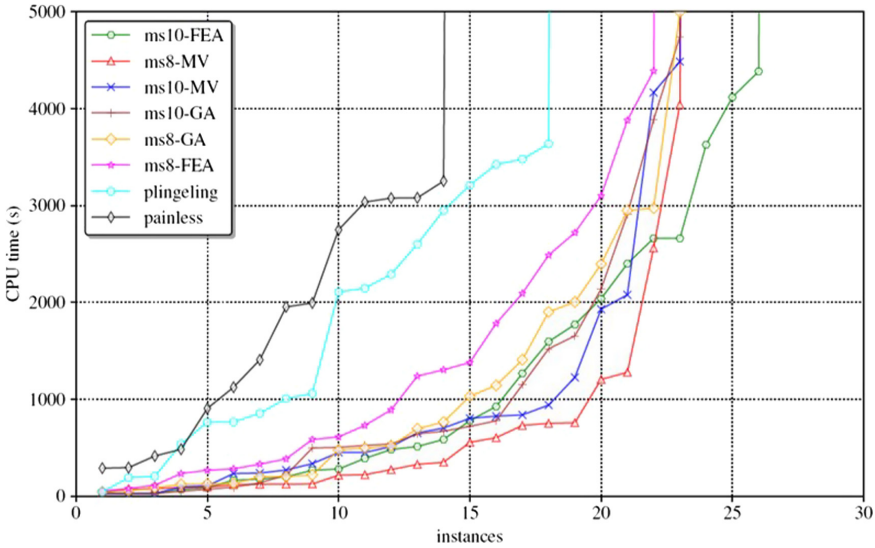

Fig. 1. Comparison of different algorithms efficiency on the inversion hash functions problems

the minimum edge weight of each customer can be increased when we delete some facilities vertices. The BLP P^{Δ} is formulated in the following manner [44].

Definition 4. *(Definition 1 from [44]). Given a tuple* $P^{\Delta} = (G(U, V, E, w), k, \Delta)$ *to find a subset* $V', V' \subseteq V$, $|V'| = k$ *such that if* V' *is deleted from* V, *the number of customers (from U), whose minimum edge weight is increased by more than* Δ, *is minimal.*

In [44] this problem was reduced to MaxSAT formulated by using so-called hard and soft clauses (see [25]). In such formulations, the set of clauses of the considered CNF is divided into two subsets: clauses, marked as 'hard' and 'soft'. Hard clauses must be satisfied and it is required to maximize the number of satisfied soft clauses under these conditions (or, which is the same thing, to minimize the number of unsatisfied soft clauses).

In our computational experiments we used the MaxSAT tests constructed in [44]. The peculiarity of this class of tests is that for each specific CNF formed by hard clauses a corresponding SUPBS $\tilde{X}$ is known. Thus, we can run Merging Variables Hill Climbing on this SUPBS. In addition, we took into account that the substitution of values of variables from $\tilde{X}$ can lead to a conflict on the set of hard clauses: in each such case, we determined the value of the objective function equal to the number of satisfied hard clauses.

For each assignment of variables from $\tilde{X}$ that satisfies all hard clauses, the value of the objective function was determined as the number of unsatisfied soft clauses and this value was minimized: this is a common form of representing results of testing MaxSAT solvers.

Author Proof

Algorithms for solving MaxSAT problems with hard and soft clause using the Merging Variables Principle are incomplete, unlike algorithms based on CDCL. These two classes of algorithms w.r.t. this variant of MaxSAT work according to completely different schemes, thus it is incorrect to compare them with each other directly. That is why we compared our algorithms for solving these cases of MaxSAT with incomplete SAT solver Loandra [4], which won the competition MaxSAT Evaluation 2019[4] among incomplete algorithms. In this series of experiments we used a personal computer of the following configuration: Intel Core i7-6700 CPU 3.40 GHz, 16 Gb RAM. It should be noted that several complete MaxSAT solvers which we used in our experiments weren't able to cope with the tests from the considered class for $k > 10$.

Table 2. Results of solving Branch Location Problem as MaxSAT

Test	ms8-GA	Loandra	Test	ms8-GA	Loandra	Test	ms8-GA	Loandra
k10_d100	8538	9826	k16_d300	14538	21830	k22_d500	22373	36099
k10_d200	6779	7197	k16_d400	12012	15653	k24_d100	45613	82174
k10_d300	5287	5287	k16_d500	10144	14325	k24_d200	39759	64742
k10_d400	4189	4189	k18_d100	25963	46157	k24_d300	34061	54062
k10_d500	3368	3368	k18_d200	21937	36753	k24_d400	30573	49746
k12_d100	11961	22247	k18_d300	18799	28302	k24_d500	27280	43962
k12_d200	9593	10474	k18_d400	15995	27131	k26_d100	53902	89965
k12_d300	7702	7845	k18_d500	13946	16530	k26_d200	47102	73874
k12_d400	6124	7301	k20_d100	31722	53121	k26_d300	40802	64648
k12_d500	5074	5074	k20_d200	27376	44737	k26_d400	36620	64986
k14_d100	15866	28993	k20_d300	23357	36773	k26_d500	32801	57051
k14_d200	12908	22168	k20_d400	20254	25355	k28_d100	62303	110191
k14_d300	10647	11344	k20_d500	18021	31686	k28_d200	55116	84828
k14_d400	8685	11286	k22_d100	38083	64766	k28_d300	48291	74623
k14_d500	7311	7556	k22_d200	33293	51940	k28_d400	43205	68754
k16_d100	20642	40328	k22_d300	28400	42631	k28_d500	38815	68217
k16_d200	17024	26589	k22_d400	25194	40265			

In Table 2 we present the results of experiments for the second class of tests (related to BLB). In all these problems $|V| = 58$. The test name `kN_dM` corresponds to the problem for $k = \mathrm{N}$ and $\Delta = \mathrm{M}$. For these experiments (based on preliminary testing), we chosen the MVP variant with $l = 8$ and the procedure for exiting local maxima based on the Genetic Algorithm. Both MVHC and Loandra were launched with a time limit of 1000 s. Each Best Known Value, presented in Table 2, is the smallest value of the objective function, which is equal to the number of unsatisfied soft clauses, found in time limit (the smaller this value, the better the found point). From Table 2 it can be seen that on

[4] https://maxsat-evaluations.github.io/2019/.

Author Proof

the considered test class the used MVHC variant always finds a point at which the value of the objective function is no worse than at the point found by the Loandra solver. For most tests, the value of the objective function found by MVHC is better (sometimes quite significant).

6 Conclusion and Future Work

In this paper we presented the results of application of the Merging Variables Principle (MVP) metaheuristic strategy to one special class of MaxSAT problem that consists in the maximization of number of satisfied clauses in an arbitrary Conjunctive Normal Form (CNF). We showed that for a considered subclass of MaxSAT the dimension of the search space is significantly less than the number of variables in CNF. We used a maximization strategy based on MVP on such a reduced space. To exit strong local maxima (this concept was introduced in [40]) we applied two evolutionary strategies, namely, $(1+1)-FEA_\beta$ described in [13] and one variant of Genetic Algorithm presented in [36]. As test sets we considered the preimage finding problems of cryptographic hash functions (so-called preimage attacks) and MaxSAT variants of the Branch Location Problem, which is a special case of the Facility Location Problem. On the considered classes of tests algorithms that are based on MVP showed higher efficiency in comparison with the algorithms that won the SAT competitions, SAT Races and MaxSAT Evaluation competitions in recent years. In the nearest future we plan to investigate the possibility of applying MVP-based computational schemes to the search problems of some combinatorial designs.

Acknowledgements. We express our deep gratitude to Dr. Oleg Zaikin for his valuable advice and help. The research was funded by Russian Science Foundation (project No. 16-11-10046).

References

1. Ahuja, R.K., Ergun, O., Orlin, J.B., Punnen, A.P.: A survey of very large-scale neighborhood search techniques. Discrete Appl. Math. **123**(1–3), 75–102 (2002)
2. Ansotegui, C., Bonet, M.L., Levy, J.: Sat-based MaxSAT algorithms. Artif. Intell. **196**, 77–105 (2013)
3. Avella, P., D'Auria, B., Salerno, S., Vasil'ev, I.: A computational study of local search algorithms for Italian high-school timetabling. J. Heuristics **13**(6), 543–556 (2007)
4. Berg, J., Demirović, E., Stuckey, P.J.: Core-boosted linear search for incomplete MaxSAT. In: Rousseau, L.-M., Stergiou, K. (eds.) CPAIOR 2019. LNCS, vol. 11494, pp. 39–56. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-19212-9_3
5. Bessiere, C., Katsirelos, G., Narodytska, N., Walsh, T.: Circuit complexity and decompositions of global constraints. In: IJCAI 2009, Proceedings of the 21st International Joint Conference on Artificial Intelligence, Pasadena, California, USA, 11–17 July 2009, pp. 412–418 (2009)

Author Proof

6. Biere, A.: CaDiCaL, lingeling, plingeling, treengeling, YalSAT entering the SAT competition 2017. In: Balyo, T., Heule, M.J.H., Järvisalo, M. (eds.) SAT Competition 2017, vol. B-2017-1, pp. 14–15 (2017)
7. Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability, vol. 185. IOS Press (2009)
8. Boros, E., Hammer, P.L.: Pseudo-boolean optimization. Discrete Appl. Math. **123**(1–3), 155–225 (2002)
9. Burke, E., Kendall, G.: Search Methodologies, 2nd edn. Springer, New York (2014). https://doi.org/10.1007/978-1-4614-6940-7
10. Chang, C.L., Lee, R.C.T.: Symbolic Logic and Mechanical Theorem Proving, 1st edn. Academic Press Inc, Orlando (1997)
11. Cook, S.A.: The complexity of theorem-proving procedures. In: Proceedings of the 3rd Annual ACM Symposium on Theory of Computing, 3–5 May 1971, Shaker Heights, Ohio, USA, pp. 151–158 (1971)
12. Davies, J., Bacchus, F.: Solving MAXSAT by solving a sequence of simpler SAT instances. In: Lee, J. (ed.) CP 2011. LNCS, vol. 6876, pp. 225–239. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-23786-7_19
13. Doerr, B., Le, H.P., Makhmara, R., Nguyen, T.D.: Fast genetic algorithms. In: Proceedings of the Genetic and Evolutionary Computation Conference. GECCO 2017, pp. 777–784. Association for Computing Machinery (2017)
14. Dowling, W.F., Gallier, J.H.: Linear-time algorithms for testing the satisfiability of propositional horn formulae. J. Log. Program. **1**(3), 267–284 (1984)
15. Droste, S., Jansen, T., Wegener, I.: On the analysis of the (1 + 1) evolutionary algorithm. Theor. Comput. Sci. **276**(1–2), 51–81 (2002)
16. Bacchus, F.: CSPs: adding structure to SAT. In: Biere, A., Gomes, C.P. (eds.) SAT 2006. LNCS, vol. 4121, pp. 10–10. Springer, Heidelberg (2006). https://doi.org/10.1007/11814948_2
17. Garey, M.R., Johnson, D.S.: Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman & Co., New York (1979)
18. Goldreich, O.: Computational Complexity: A Conceptual Perspective, 1st edn. Cambridge University Press, New York (2008)
19. Gu, J., Purdom, P.W., Franco, J., Wah, B.W.: Algorithms for the satisfiability (sat) problem: a survey. In: DIMACS Series in Discrete Mathematics and Theoretical Computer Science, pp. 19–152. American Mathematical Society (1996)
20. Ignatiev, A., Morgado, A., Manquinho, V., Lynce, I., Marques-Silva, J.: Progression in maximum satisfiability. In: Proceedings of the Twenty-First European Conference on Artificial Intelligence. ECAI 2014, pp. 453–458. IOS Press (2014)
21. Ignatiev, A., Morgado, A., Marques-Silva, J.: RC2: an efficient MaxSAT solver. J. Satisf. Boolean Model. Comput. **11**(1), 53–64 (2019)
22. Ignatiev, A., Morgado, A., Marques-Silva, J.: PySAT: a python toolkit for prototyping with SAT oracles. In: Beyersdorff, O., Wintersteiger, C.M. (eds.) SAT 2018. LNCS, vol. 10929, pp. 428–437. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-94144-8_26
23. Laporte, G., Nickel, S., Saldanha da Gama, F. (eds.): Location Science. Springer, Switzerland (2015). https://doi.org/10.1007/978-3-319-13111-5
24. Le Frioux, L., Baarir, S., Sopena, J., Kordon, F.: PaInleSS: a framework for parallel SAT solving. In: Gaspers, S., Walsh, T. (eds.) SAT 2017. LNCS, vol. 10491, pp. 233–250. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-66263-3_15
25. Li, C.M., Manyà, F.: MaxSAT, hard and soft constraints. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications, vol. 185, pp. 613–631. IOS Press (2009)

Author Proof

26. Luke, S.: Essentials of Metaheuristics, 2nd edn. George Mason University (2015)
27. MacWilliams, F., Sloane, N.: The Theory of Error-Correcting Codes. North Holland (1983)
28. Marques-Silva, J.P., Lynce, I., Malik, S.: Conflict-driven clause learning SAT solvers. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications, vol. 185, pp. 131–153. IOS Press (2009)
29. Mendelson, E.: Introduction to Mathematical Logic, 4th edn. Chapman and Hall (1997)
30. Menezes, A.J., Vanstone, S.A., Oorschot, P.C.V.: Handbook of Applied Cryptography, 1st edn. CRC Press Inc, Boca Raton (1996)
31. Mladenović, N., Hansen, P.: Variable neighborhood search. Comput. Oper. Res. **24**(11), 1097–1100 (1997)
32. Morgado, A., Heras, F., Liffiton, M.H., Planes, J., Marques-Silva, J.: Iterative and core-guided MaxSAT solving: a survey and assessment. Constraints Int. J. **18**(4), 478–534 (2013)
33. Mühlenbein, H.: How genetic algorithms really work: mutation and hillclimbing. In: Parallel Problem Solving from Nature 2, PPSN-II, 28–30 September 1992, Brussels, Belgium, pp. 15–26 (1992)
34. Nakamoto, S.: Bitcoin: a peer-to-peer electronic cash system (2019)
35. Narodytska, N., Bacchus, F.: Maximum satisfiability using core-guided MaxSAT resolution. In: Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence. AAAI 2014, pp. 2717–2723. AAAI Press (2014)
36. Pavlenko, A., Semenov, A., Ulyantsev, V.: Evolutionary computation techniques for constructing SAT-based attacks in algebraic cryptanalysis. In: Kaufmann, P., Castillo, P.A. (eds.) EvoApplications 2019. LNCS, vol. 11454, pp. 237–253. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-16692-2_16
37. Russell, S.J., Norvig, P.: Artificial Intelligence - A Modern Approach, Third International Edition. Pearson Education (2010)
38. Semenov, A.A.: Decomposition representations of logical equations in problems of inversion of discrete functions. J. Comput. Syst. Sci. Int. **48**, 718–731 (2009)
39. Semenov, A., Otpuschennikov, I., Gribanova, I., Zaikin, O., Kochemazov, S.: Translation of algorithmic descriptions of discrete functions to SAT with applications to cryptanalysis problems. Logical Methods Comput. Sci. **16**(1), 1–42 (2020)
40. Semenov, A.A.: Merging variables: one technique of search in pseudo-boolean optimization. In: Mathematical Optimization Theory and Operations Research. Communications in Computer and Information Science, vol. 1090, pp. 86–102 (2019)
41. Tseitin, G.S.: On the complexity of derivation in propositional calculus. In: Studies in Constructive Mathematics and Mathematical Logic, part II, Seminars in Mathematics, pp. 115–125 (1970)
42. Wegener, I.: Theoretical aspects of evolutionary algorithms. In: Orejas, F., Spirakis, P.G., van Leeuwen, J. (eds.) ICALP 2001. LNCS, vol. 2076, pp. 64–78. Springer, Heidelberg (2001). https://doi.org/10.1007/3-540-48224-5_6
43. Williams, R., Gomes, C.P., Selman, B.: Backdoors to typical case complexity. In: The 18th International Joint Conference on Artificial Intelligence (IJCAI 2003), pp. 1173–1178 (2003)
44. Zaikin, O., Ignatiev, A., Marques-Silva, J.: Branch location problem with maximum satisfiability. In: ECAI 2020 (2020, in press)

Author Proof

Author Queries

Chapter 29

Query Refs.	Details Required	Author's response
AQ1	This is to inform you that corresponding author has been identified as per the information available in the Copyright form.	